Location Based Services

Definition:

Location Based Services (LBS) are mobile services that use the geographical location of a device to provide information or services to the user.

Components of LBS

1. **Location Manager – Gets the device location.**
2. **Location Provider – Provides location data (GPS, Network).**
3. **Application – Uses the location data to provide service.**

Finding Current Location

Android uses LocationManager API to obtain current device location using GPS or network provider.

Example methods:

- `getLastKnownLocation()`
- `requestLocationUpdates()`

Listening for Location Changes

Applications can listen for location changes using LocationListener.

Important methods:

- **onLocationChanged()**
- **onProviderDisabled()**
- **onProviderEnabled()**

Proximity Alerts

Proximity alerts notify the user when the device enters or leaves a specific geographic area.

Example: **A shopping app sends a notification when the user comes near a mall.**

Working with Google Maps

Google Maps API allows developers to:

- **Display maps in apps**
- **Add markers and routes**
- **Show user location**

Real-life Example

Food delivery apps like Zomato or Swiggy use LBS to track delivery location.

Conclusion

LBS helps mobile applications provide services based on the real-time location of users.

Showing Google Map in an Activity

1. Definition: Showing Google Map in an Activity means displaying an interactive Google Map inside an Android application screen using the Google Maps API.

2. Requirements

- Google Maps API Key
- Google Play Services Library
- Internet permission in Android Manifest

Example:
```
<uses-permission android:name="android.permission.INTERNET"/>
```

3. Steps to Display Google Map in an Activity

Step 1: Generate Google Maps API Key

Create an API key from Google Cloud Console and enable Google Maps API.

Step 2: Add Map Fragment in Layout

```
<fragment
    android:id="@+id/map"
    android:name="com.google.android.gms.maps.SupportMapFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Step 3: Initialize Map in Activity

```
GoogleMap mMap;

SupportMapFragment mapFragment =
(SupportMapFragment) getSupportFragmentManager()
.findFragmentById(R.id.map);
mMap = mapFragment.getMap();
```

Step 4: Add Marker on Map

```
LatLng location = new LatLng(21.1702, 72.8311);
mMap.addMarker(new
MarkerOptions().position(location).title("Surat"));
mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(location,10));
```

4. Features of Google Map in Activity

- Display current location
- Add markers for places
- Zoom in and zoom out
- Show routes and directions

5. Example **:** A tourism application shows famous places on Google Map so users can easily find locations and navigate there.

Conclusion: Using Google Maps API, Android applications can embed maps inside activities and provide location-based services such as navigation, place search, and route display.

Map Overlays

1. Definition: **Map Overlay is a layer that is placed on top of a Google Map to display additional information such as markers, images, lines, or shapes.**

2. Purpose of Map Overlays

- **To show extra information on the map.**
- **To mark important locations or places.**
- **To display routes, areas, or boundaries.**

3. Types of Map Overlays

1. **Marker Overlay – Displays location markers on the map.**
2. **Shape Overlay – Draws shapes like circle, polygon, or rectangle.**
3. **Polyline Overlay – Used to display routes or paths between locations.**
4. **Image Overlay – Displays an image on the map.**

4. Working of Map Overlay

1. **Create an overlay object.**
2. **Add location or graphical data.**
3. **Display it on the Google Map.**

5. Example Code (Concept)

```
MarkerOptions marker = new MarkerOptions()
.position(new LatLng(21.1702,72.8311))
.title("Surat");

map.addMarker(marker);
```

This code adds a marker overlay at the specified location on the map.

6. Advantages

- **Improves map visualization.**
- **Helps users easily identify locations.**
- **Allows graphical representation of geographic data.**

7. Example **: In a restaurant finder application, map overlays are used to display multiple restaurant locations as markers on Google Map.**

Conclusion: **Map overlays enhance Google Maps by allowing applications to display markers, shapes, routes, and images, making maps more interactive and informative.**

Itemized overlays

1. Definition: Itemized Overlay is a class used in Android Google Maps to display multiple overlay items (markers) on a map. Each item represents a specific location with information such as title and description.

2. Purpose of Itemized Overlays

- To manage multiple markers on the map.
- To display information for each location.
- To make maps more interactive for users.

3. Components of Itemized Overlay

1. OverlayItem – Represents a single location marker.
2. ItemizedOverlay Class – Manages a list of overlay items.
3. Drawable Marker – Icon used to represent the location.

4. Working of Itemized Overlay

1. Create a list of overlay items.
2. Add location information (latitude and longitude).
3. Display all markers on the map through ItemizedOverlay.

5. Example Code (Concept)

```
GeoPoint point = new GeoPoint(lat, lon);
OverlayItem item = new OverlayItem(point, "Place Name",
"Description");
overlayList.add(item);
```

This code creates an overlay item and adds it to the list of markers.

6. Advantages

- Displays multiple locations efficiently.
- Each marker can show additional information when clicked.
- Improves map usability.

7. Example: In a hotel booking application, itemized overlays are used to show multiple hotel locations on the map, allowing users to select a hotel and view details.

Conclusion: Itemized overlays help Android applications display and manage multiple location markers on Google Maps, making map-based applications more informative and user-friendly.

Geocoder

Geocoder is an Android class used to convert addresses into geographic coordinates (latitude and longitude) and vice versa.

2. Types of Geocoding

1. Forward Geocoding: Converts an address into latitude and longitude.

Example: "Delhi, India" → (28.6139, 77.2090)

2. Reverse Geocoding
Converts latitude and longitude into a readable address.

Example: (28.6139, 77.2090) → Delhi, India

3. Geocoder Class: Android provides the Geocoder class in the location package to perform geocoding operations.

Important methods:

- **getFromLocation() – Converts coordinates to address**
- **getFromLocationName() – Converts address to coordinates**

4. Example Code (Concept)

```
Geocoder geocoder = new Geocoder(this, Locale.getDefault());

List<Address> addresses =
geocoder.getFromLocationName("Surat", 1);

Address address = addresses.get(0);
double latitude = address.getLatitude();
double longitude = address.getLongitude();
```

5. Uses of Geocoder

- **Finding location coordinates from address**
- **Displaying address from GPS location**
- **Used in navigation and map applications**

6. Example

In a taxi booking app, when a user enters a pickup address, the Geocoder converts the address into latitude and longitude to show the location on Google Map.

Conclusion

Geocoder is an important Android class that helps applications convert addresses and geographic coordinates, making location-based services easier to implement.

Displaying route on map

1. Definition: Displaying route on map means showing the path between two locations (source and destination) on Google Map using Android application.

2. Purpose

- To provide navigation path to users
- To show shortest or optimal route
- To display distance and travel time

3. Steps to Display Route

1. **Get Source and Destination**
   Obtain latitude and longitude of both locations.
2. **Use Google Directions API**
   Send request to Google Directions API to get route data.
3. **Receive Route Data**
   Data is returned in JSON format.
4. **Parse JSON Data**
   Extract route points (coordinates).
5. **Draw Route on Map**
   Use Polyline to display route.

4. Example Code (Concept)

```
PolylineOptions polylineOptions = new PolylineOptions()
.add(new LatLng(21.1702,72.8311))   // Source
.add(new LatLng(23.0225,72.5714));  // Destination

map.addPolyline(polylineOptions);
```

5. Features

- Shows route path visually
- Supports multiple routes
- Can display distance and estimated time

6. Example: Navigation apps like Google Maps display route from user's current location to destination, helping users reach faster.

Conclusion: Displaying route on map helps Android applications provide efficient navigation by visually representing the path between two locations using polyline on Google Map.

Drawing on screen – using canvas

1. Definition: **Drawing on screen in Android is done using Canvas and Paint classes, which allow developers to draw shapes, text, and graphics on the screen.**

2. Canvas Class: **Canvas is used to draw objects on the screen.**

Functions of Canvas:

- **Draw shapes (circle, rectangle, line)**
- **Draw text**
- **Draw images (bitmap)**

3. Paint Class: **Paint defines style and color of drawing.**

Properties of Paint:

- **Color**
- **Stroke width**
- **Style (fill or stroke)**
- **Text size**

4. Working Process

1. **Create a custom View class**
2. **Override onDraw() method**
3. **Use Canvas and Paint inside it to draw objects**

5. Example Code

```
public class MyView extends View {
    Paint paint = new Paint();
    public MyView(Context context) {
        super(context);
    }
    @Override
    protected void onDraw(Canvas canvas) {
        paint.setColor(Color.RED);
        paint.setStrokeWidth(5);
        canvas.drawCircle(200, 200, 100, paint);
        canvas.drawText("Hello", 100, 100, paint);
    }
}
```

6. Uses

- **Drawing shapes and graphics**
- **Creating custom UI components**
- **Game development**

7. Example: **A drawing application allows users to draw shapes or sketches using Canvas and Paint.**

Conclusion: **Canvas and Paint are essential classes in Android used to draw graphics, shapes, and text on the screen, enabling custom UI and graphic-based applications.**

Working with bitmap, shapes

1. Definition: In Android graphics, Bitmap and Shapes are used to display and draw images and geometric figures on the screen.

2. Bitmap: Bitmap is an object that represents an image (pixel-based graphic).

Features

- Stores image in pixels
- Used for displaying photos or icons
- Can be drawn on Canvas

Common Methods

`BitmapFactory.decodeResource()` – Load image
`canvas.drawBitmap()` – Draw image on screen

Example Code :

```
Bitmap bitmap = BitmapFactory.decodeResource(getResources(),
R.drawable.image);
canvas.drawBitmap(bitmap, 100, 100, null);
```

3. Shapes: Shapes are geometric figures like rectangle, circle, line, etc., drawn using Canvas and Paint.

Types of Shapes

- Circle
- Rectangle
- Line
- Oval

Example Code

```
Paint paint = new Paint();
paint.setColor(Color.BLUE);

canvas.drawRect(100, 100, 300, 300,
paint);
canvas.drawCircle(200, 200, 50,
paint);
```

4. Difference between Bitmap and Shapes

Bitmap	Shapes
Pixel-based image	Mathematical drawing
Used for photos/icons	Used for geometric figures
Loaded from resources	Created using Canvas

5. Uses

- Bitmap → Display images (gallery, icons)
- Shapes → Custom UI, games, graphics

6. Example: A game application uses bitmap for character images and shapes for drawing boundaries or objects. ***Conclusion:*** Bitmap and Shapes are important in Android graphics for displaying images and drawing geometric figures, helping in building interactive and visually rich applications.

2D Animation - Drawable, View, Property animation 1

1. Definition: **2D Animation in Android is used to create motion or visual effects for UI elements such as images, buttons, or layouts.**

2. Types of 2D Animation

1. Drawable Animation (Frame Animation): **Drawable animation displays a sequence of images frame by frame to create animation.**

Features:

- **Uses multiple images**
- **Defined in XML**
- **Works like a slideshow**

Example (Concept)

```
<animation-list>
   <item android:drawable="@drawable/img1" android:duration="200"/>
   <item android:drawable="@drawable/img2" android:duration="200"/>
</animation-list>
```

Example Use
Loading animation or cartoon animation.

2. View Animation (Tween Animation): **View animation is used to apply animation effects on views like rotate, scale, translate, fade.**

Types

- **Translate (move)**
- **Rotate**
- **Scale**
- **Alpha (fade in/out)**

```
<rotate
    android:fromDegrees="0"
    android:toDegrees="360"
    android:duration="1000"/>
```

Example Use **: Rotating image or moving button.**

3. Property Animation **: Property animation changes actual properties of objects such as position, size, rotation.**

Features

- **More flexible than view animation**
- **Can animate any object property**
- **Uses ObjectAnimator / ValueAnimator**

Example Code

```
ObjectAnimator anim = ObjectAnimator.ofFloat(view, "rotation", 0f, 360f);
anim.setDuration(1000);
anim.start();
```

Example Use: **Smooth UI transitions and complex animations.**

3. Difference Between Animations

Drawable	View	Property
Frame-based	Predefined effects	Property-based
Uses images	Works on views	Works on objects
Less flexible	Moderate	Highly flexible

---

4. Uses

- **Improve UI experience**
- **Create interactive applications**
- **Used in games and mobile apps**

5. Example : **A music player app uses animation to rotate album image while playing music.**

Conclusion : **Android provides Drawable, View, and Property animations to create smooth and interactive 2D animations, improving user experience and visual appeal of applications.**

2D Animation - Drawable, View, Property animation 2

1. Definition: Signing certificate is used to secure and identify an Android application, and APK/Bundle are the final installable files generated for distribution.

2. Signing Certificate: A signing certificate is a digital key used to sign an Android app, ensuring authenticity and security.

Purpose

- **Identifies the developer**
- **Prevents unauthorized modification**
- **Required for publishing on Google Play**

Types

1. Debug Certificate – Used during development.

2. Release Certificate – Used for publishing app

3. Generating APK (Android Package Kit): APK is a file format used to install Android applications.

Steps to Generate APK

1. **Open project in Android Studio**
2. **Go to Build → Generate Signed APK**
3. **Create or select keystore**
4. **Enter key details (password, alias)**
5. **Build APK**

Output: A .apk file is generated for installation.

4. Generating Android App Bundle (AAB): Android App Bundle is a publishing format that contains all app resources, and Google Play generates optimized APKs for different devices.

Steps to Generate Bundle

1. **Go to Build → Generate Signed Bundle / APK**
2. **Select Android App Bundle**
3. **Choose keystore**
4. **Build bundle**

Output A .aab file is generated.

5. Difference Between APK and Bundle

APK	App Bundle (AAB)
Direct install file	Not directly installable
Larger size	Smaller optimized size
Contains all resources	Play Store generates APKs

6. Example: **When a developer finishes an app, they sign it using a release key and generate an AAB file, which is uploaded to Google Play for users to download.**

Conclusion: **Signing certificate ensures security and authenticity, while APK and App Bundle are used to package and distribute Android applications efficiently.**

1. Definition: Distributing an Android app via Google Play means publishing the application on Google Play Store so users can download and install it.

2. Requirements:

- **Google Play Developer Account**
- **Signed APK or App Bundle (AAB)**
- **App details (name, description, screenshots, icon)**

3. Steps to Distribute App

Step 1: Create Developer Account

Register on Google Play Console (one-time fee required).

Step 2: Prepare App for Release

- **Sign the app with release certificate**
- **Generate APK or AAB file**

Step 3: Create New Application

- **Enter app name**
- **Select language**

Step 4: Upload APK / AAB

Upload the generated file to Google Play Console.

Step 5: Add App Details

- **App description**
- **Screenshots**
- **App icon**
- **Category (education, game, etc.)**

Step 6: Set Pricing and Distribution

- **Free or paid**
- **Select countries/regions**

Step 7: Publish App

Submit app for review and approval by Google.

4. Features of Google Play Distribution

- **Automatic app updates**
- **Wide user reach**
- **Security checks by Google**
- **Device compatibility support**

5. Example

A developer creates a shopping app, uploads its AAB file on Google Play, adds details and publishes it so users worldwide can download it.

Conclusion

Google Play Store provides a platform to publish, manage, and distribute Android applications globally, making it easy for developers to reach a large audience.

Obfuscating and optimizing with ProGuard

1. Definition: **ProGuard is a tool used in Android to shrink, optimize, and obfuscate application code to improve performance and security.**

2. Purpose of ProGuard

- **Reduce APK size**
- **Improve performance**
- **Protect code from reverse engineering**

3. Features of ProGuard

1. **Code Shrinking:** **Removes unused classes, methods, and variables.**
2. **Optimization:** **Improves code efficiency by simplifying bytecode.**
3. **Obfuscation:** **Renames classes, methods, and variables into unreadable names.**
   Example: `calculateTotal() → a()`
4. **Preverification:** **Ensures code is valid for execution.**

4. Working of ProGuard

1. **Analyze application code**
2. **Remove unused code**
3. **Optimize remaining code**
4. **Obfuscate names**
5. **Generate optimized APK**

5. Configuration File

ProGuard rules are defined in a file: `proguard-rules.pro`

Example Rule: **-keep public class * extends android.app.Activity**

6. Advantages

- **Smaller APK size**
- **Faster app performance**
- **Increased code security**
- **Harder to reverse engineer**

7. Example: **In a banking application, ProGuard is used to hide sensitive logic and reduce app size, making it more secure.**

Conclusion: **ProGuard is an important tool that helps Android developers optimize and secure applications by shrinking and obfuscating code, improving both performance and protection.**